❹ Creates a DbContext and passes in the options

❺ Ensures that the in-memory database matches EF Core's model (similar to running migrations)

❻ Adds some recipes to the DbContext

❼ Saves the changes to the in-memory database

❽ Creates a fresh DbContext to test that you can retrieve data from the DbContext

❾ Creates the Recipe-Service to test and pass in the fresh DbContext

❿ Executes the GetRecipe function. This executes the query against the in-memory database.

⓫ Verifies that you retrieved the recipe correctly from the in-memory database

This example follows the standard format for any time you need to test a class that depends on an EF Core `DbContext`:

1. Create a `SqliteConnection` with the `"DataSource=:memory:"` connection string, and open the connection.
2. Create a `DbContextOptionsBuilder<>` and call `UseSqlite()`, passing in the open connection.
3. Retrieve the `DbContextOptions` object from the `Options` property.
4. Pass the options to an instance of your `DbContext` and ensure the database matches EF Core's model by calling `context.Database.EnsureCreated()`. This is similar to running migrations on your database, but it should be used only on test databases. Create and add any required test data to the in-memory database, and call `SaveChanges()` to persist the data.
5. Create a new instance of your `DbContext` and inject it into your test class. All queries will be executed against the in-memory database.